Software Architecture Models: Monolithic, N-Tier, and Microservices Models

structure, trade-offs, and evolution

SEAtS | 188398

University of Southampton

Intended Learning Outcomes

By the end of this lecture you will be able to:

1. Describe the defining characteristics of monolithic, N-tier, and microservices architectures.
2. Compare the strengths, weaknesses, and ideal use cases for each model.
3. Explain how architectural evolution influences deployment, scalability, and maintenance.

© University of Southampton

Architecture as System Structure

In software engineering, architecture is the fundamental organisation of a system—its components, relationships, and guiding principles.

Architecture articulates a strategic balance between **delivery speed**, **operational stability**, and **future evolution**. We explore this balance by contrasting three widely adopted paradigms.

© University of Southampton

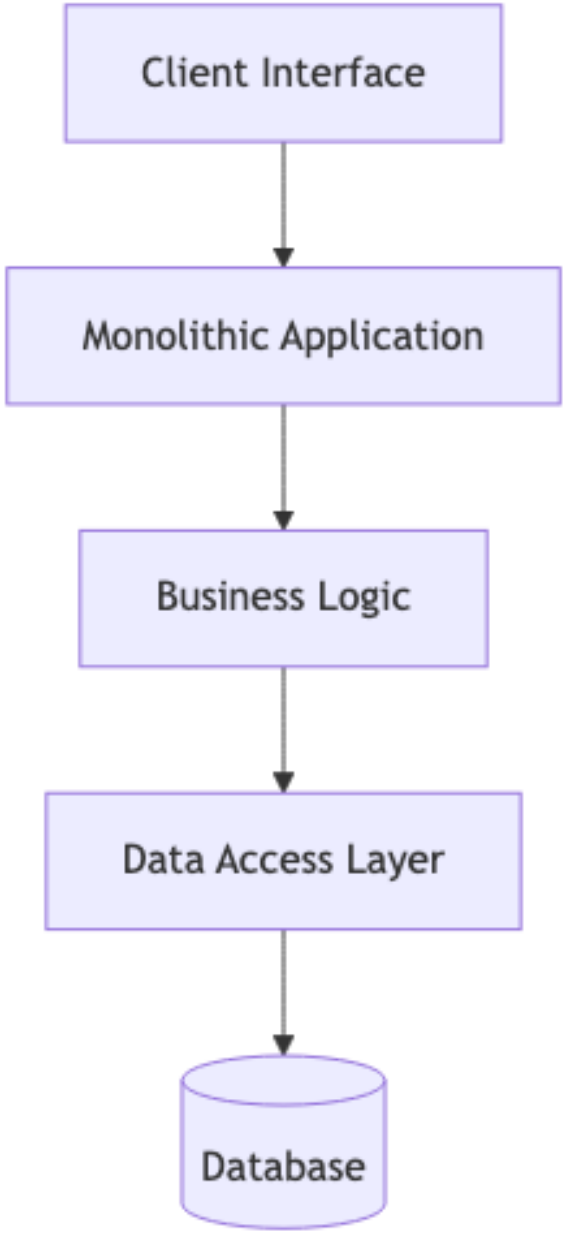
Evolution of Enterprise Architecture

Software architecture evolution reflects a move from tight coupling toward modular independence.

Monolithic (pre-2000s)	N-tier (2000s)	Microservices (2010s-present)
Unified codebases, single deployment artefacts.	Layered separation of presentation, application logic, and data.	Distributed services with independent deployment.

© University of Southampton

Monolithic Architecture



A **monolithic architecture** packages all functionality—user interface, business logic, and data access—into a single executable. Deployment and scaling are performed as one unit.

© University of Southampton

Monolithic Architecture

Advantages

- Simple initial development and deployment.
- Straightforward debugging within a single codebase.
- High performance through in-process communication.

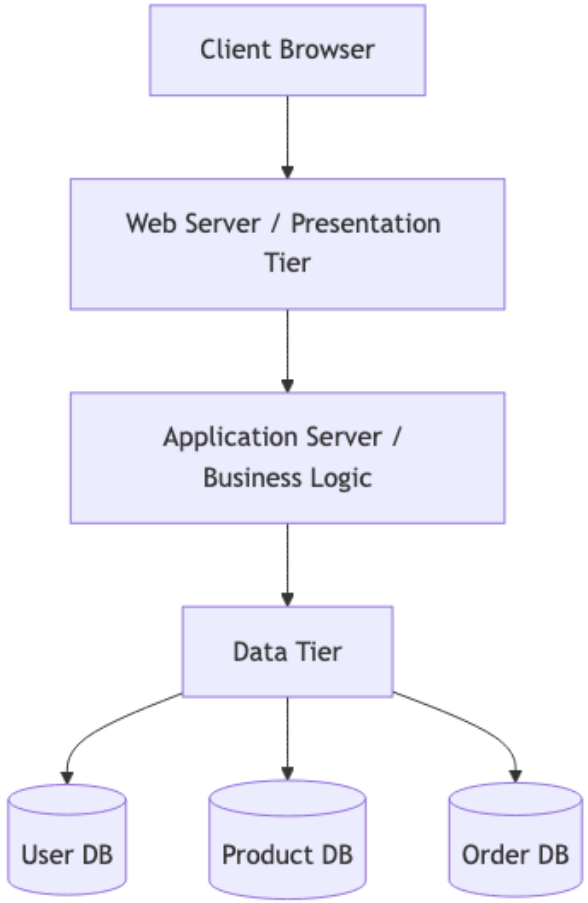
Disadvantages

- Limited horizontal scalability—the entire application must scale.
- Redeploying any change requires redeploying the whole system.
- Technology lock-in around a single language and framework.

Example: Early e-commerce platforms and legacy ERP suites.

© University of Southampton

N-Tier Architecture



N-tier (layered) architecture separates responsibilities across tiers, typically presentation, application, and data. Layers interact via well-defined interfaces, often REST or RPC APIs.

© University of Southampton

N-Tier Architecture

Advantages

- Separation of concerns improves maintainability and testing.
- Individual tiers can scale independently.
- Facilitates integration via APIs and middleware.

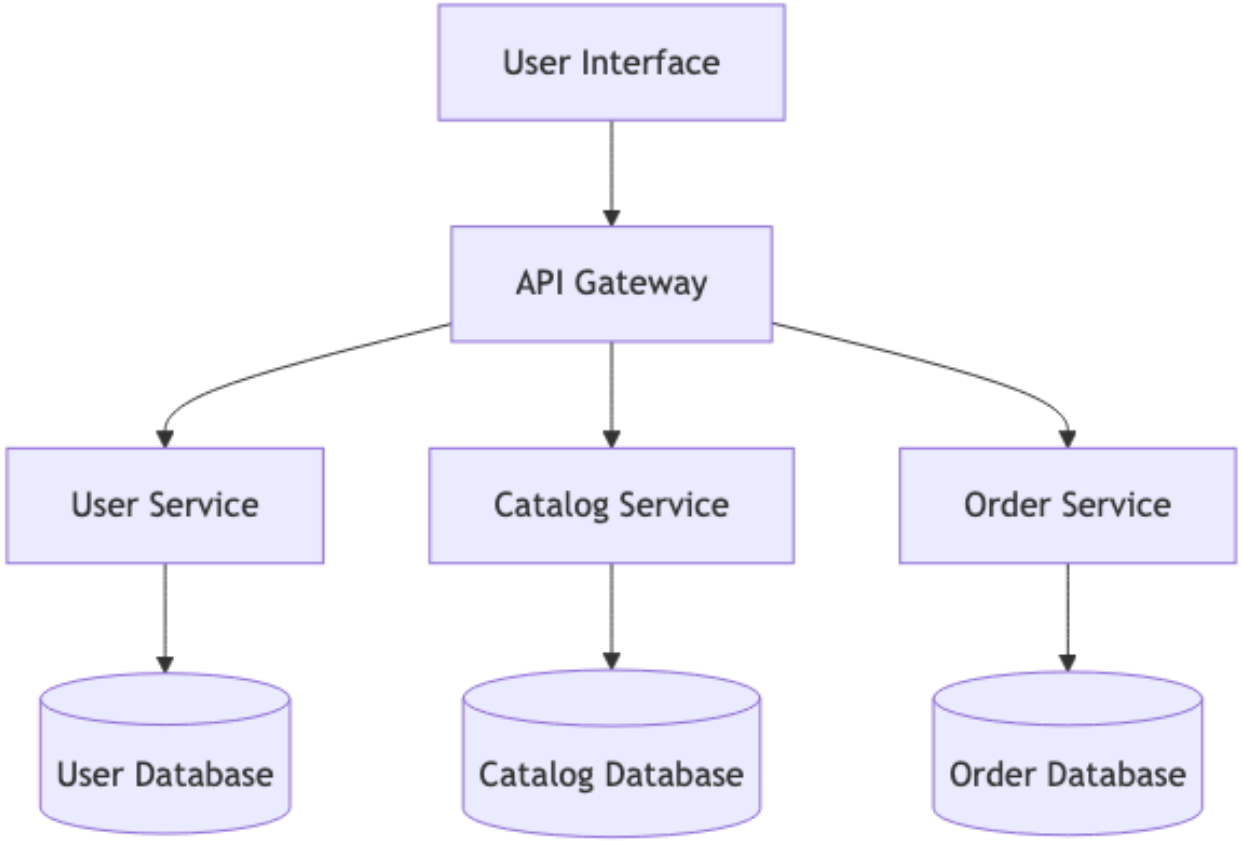
Disadvantages

- Network hops introduce latency and complexity.
- Coordination required between teams managing different tiers.
- Application tier can still become a scaling bottleneck.

Example: Online banking portals built with Java EE or .NET MVC.

© University of Southampton

Microservices Architecture



Microservices architecture decomposes an application into independently deployable services, each encapsulating a bounded context. Services communicate via synchronous APIs or asynchronous messaging.

© University of Southampton

Microservices Architecture

Advantages

- Independent development, deployment, and scaling per service.
- Fault isolation—individual service failures need not topple the system.
- Technology heterogeneity—teams choose the best-fit stack.
- Enables continuous delivery and rapid experimentation.

Disadvantages

- Distributed systems complexity: latency, consistency, monitoring, security.
- Requires mature DevOps, CI/CD, and orchestration platforms.
- Higher operational overhead, especially for smaller organisations.

Example: Netflix, Amazon, and Spotify underpin their platforms with microservices.

© University of Southampton

Comparative View

Feature	Monolithic	N-Tier	Microservices
Deployment	Single artefact	Multiple coordinated tiers	Independent services
Scalability	Limited	Moderate, tier-based	Fine-grained, service-level
Maintainability	Low as apps grow	Moderate	High with strong governance
Complexity	Low initially	Medium	High operationally
Best For	Simple, stable apps	Enterprise web workloads	Large, cloud-native ecosystems

© University of Southampton

Transitioning in Practice

Enterprises rarely jump directly from monolith to microservices. Common path:

1. Refactor monolith into modular N-tier or “modulith” structure.
2. Identify bounded contexts where autonomy delivers value.
3. Gradually extract services, backed by automation and observability.

Case example: Amazon evolved from a monolithic retail platform to service-oriented architecture before embracing microservices (Newman, 2021).

© University of Southampton

Selection Criteria

When choosing an architecture, assess:

1. **System complexity** – microservices add overhead unless scale demands it.
2. **Team size and autonomy** – small teams benefit from simpler deployments.
3. **Scalability requirements** – evaluate load patterns and bottlenecks.
4. **Operational maturity** – CI/CD pipelines, monitoring, incident response.
5. **Data integrity needs** – monoliths simplify ACID transactions.

© University of Southampton

Design Principles Across Models

Regardless of architecture, adhere to:

- **Separation of concerns** – isolate distinct responsibilities.
- **Loose coupling** – minimise cross-component dependencies.
- **High cohesion** – keep related functionality together.
- **Clear contracts** – document APIs and service agreements.
- **Automation & testing** – use CI/CD and observability to preserve architectural integrity.

These principles underpin sustainable enterprise software.

© University of Southampton

Closing Thought

“Architecture is the art of determining what’s important in a system and suppressing everything else.” — David Parnas (1972). Skilled architects balance simplicity, flexibility, and scalability to match organisational maturity.

© University of Southampton

Summary

- Monolithic architectures favour simplicity but limit scaling and agility.
- N-tier architectures introduce separation of concerns without fully decoupling teams.
- Microservices deliver fine-grained scalability at the cost of operational complexity.
- Choose architectures based on system complexity, organisational maturity, and measurable outcomes.

© University of Southampton

References

- Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley.
- Kruchten, P. (1995). Architectural blueprints—The 4 + 1 view model of software architecture. *IEEE Software, 12(6).*
- Newman, S. (2021). *Building Microservices* (2nd ed.). O'Reilly Media.
- TatvaSoft. (2022). Enterprise application architecture and its types. Retrieved from https://www.tatvasoft.com/outsourcing/2022/10/enterprise-application-architecture.html
- ISO/IEC/IEEE 42010:2011. Systems and software engineering—Architecture description.

© University of Southampton



© University of Southampton